VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

SideSeat: System Overview, Technology Stack & Simulation Manual

VIT Central Library Real-Time Individual Seat Allocation & Recommendation System

Course: ISWE202 - Data Structures & Algorithms	**Faculty:** Dr. P C Kishoreraja
Repository: github.com/alankrishna-pu/VITlibs	**Runtime Port:** http://localhost:8080/
Core Backend: Pure C (C11) POSIX Sockets	**Presentation:** Vanilla HTML5 / CSS3 / ES6 JS

1. How the Project Works (End-to-End Operational Flow)

SideSeat is an autonomous, high-performance library seat management utility designed to eliminate physical wandering and seat hunting in the VIT Central Library. The complete operational lifecycle operates as follows:

• **Step 1 — Server Initialization & Data Loading:** On startup, the embedded C server executes seat_init(), reading data/seats.csv into contiguous memory (g_seats[256]). Simultaneously, it indexes all 112 seats across 5 floors into a 101-bucket **DJB2 Hash Table** with Separate Chaining for $O(1)$ instant key lookups.

• **Step 2 — Client Gateway & First-Open Authentication:** When a student or librarian opens http://localhost:8080/, the application checks local session storage. If unauthenticated, the **Login Gate** immediately overlays the screen, offering one-click demo credentials for Student (21BCE1024) or Librarian (LIB-8842).

• **Step 3 — Interactive Map Exploration:** Once signed in, the client receives authoritative floor metadata. The student can click through floors (Ground to 4th Floor) or filter for specific seat categories—specifically **Side Seats** (window counters with wall power outlets) and **Corner Seats** (deep-focus study carrels).

• **Step 4 — Real-Time Availability & Walk-In Guidance:** Clicking any seat renders the **Seat Details Panel**. If the seat is VACANT, the student receives walking instructions (Floor, Zone, Orientation). If OCCUPIED, SideSeat automatically triggers a 4-tier recommendation heuristic that suggests alternative vacant seats nearby.

• **Step 5 — Librarian Seat Management:** When logged in as Librarian, staff gain interactive controls to toggle seat statuses (AVAILABLE ↔ OCCUPIED ↔ UNAVAILABLE). Each update sends an HTTP PATCH request to the C backend, instantly modifying both in-memory structures and saving to data/seats.csv.

2. Full Technology Stack Specifications

Component Layer	Technology Used	Role and Architectural Function
Core Backend	Standard C (C11)	Ultra-low-latency in-memory data structures, authoritative array, and DJB2 hash engine. Zero memory bloat.
Web Server	POSIX Sockets (sys/socket.h)	Lightweight embedded HTTP server running on TCP port 8080 with non-blocking select() I/O multiplexing.
Communications	REST API over HTTP/1.1	JSON payload exchange with Content-Length body buffering and Cross-Origin Resource Sharing (CORS).
Presentation UI	Vanilla HTML5 & CSS3	Modern library dashboard with architectural blueprint grid, color-coded seats, dark navy sidebar, and CSS flex/grid layout.
Client Logic	Vanilla JavaScript (ES6+)	Observer pattern state management, asynchronous Fetch API layer, DOM rendering without external frameworks.
Data Storage	Structured CSV (seats.csv)	Human-readable, zero-database persistence format ensuring rapid crash recovery and external auditability.
Build & Test	GNU Make & Clang	Automated compilation target (make all) and 24-assertion comprehensive DSA test runner (make test).

3. Architectural Block Diagram & Data Flow

Figure 1 illustrates the four-tier architectural model of SideSeat, detailing how requests flow seamlessly from the browser frontend through the POSIX C socket layer into the core data structures.

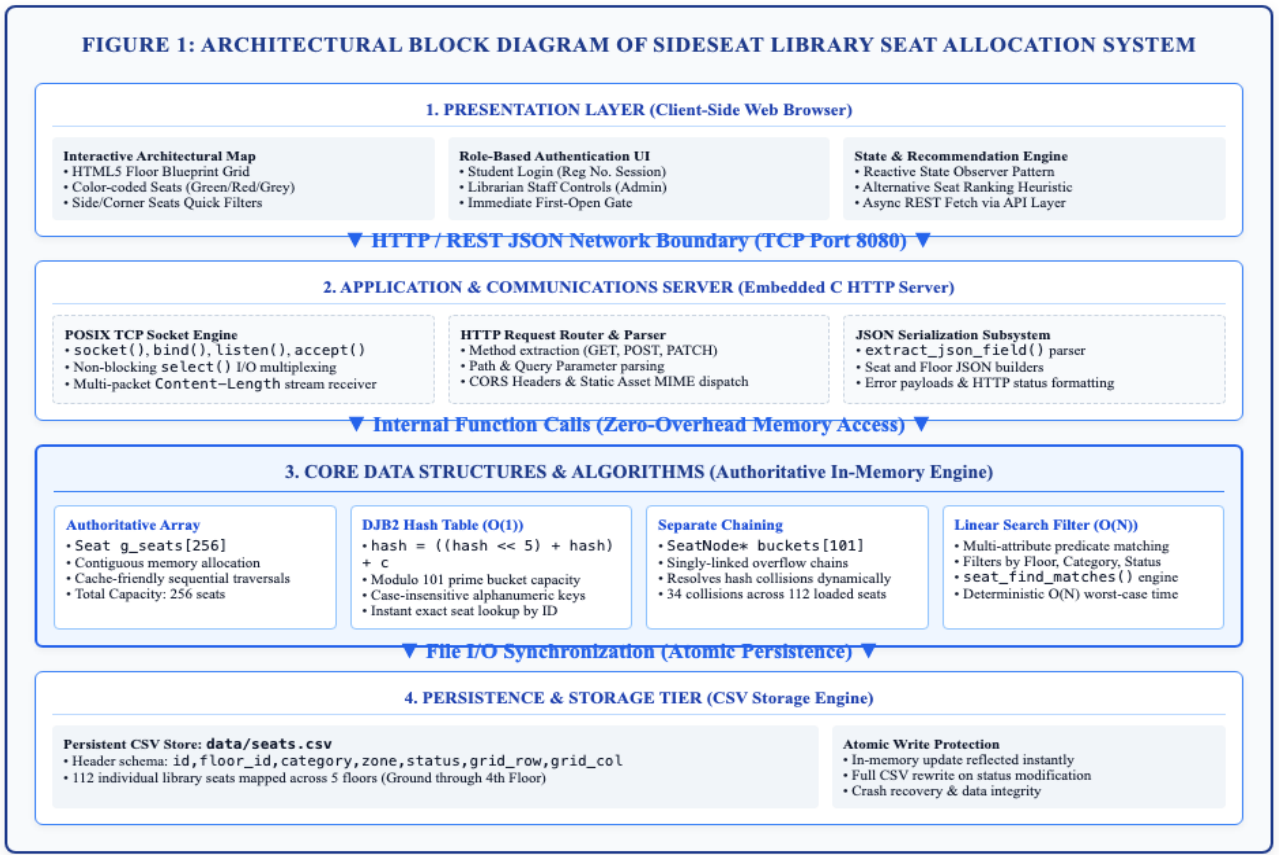


Figure 1: SideSeat End-to-End System Architecture (Presentation ↔ Sockets ↔ DSA Engine ↔ CSV Storage).

4. Core Data Structures & Algorithmic Mechanics

SideSeat implements a hybrid algorithmic design that pairs $O(1)$ constant-time key mapping with deterministic $O(N)$ multi-attribute linear search filtering:

A. DJB2 Separate Chaining Hash Table (Exact Seat ID Lookup — O(1)):

When students search for a specific seat by identifier (e.g. G-S01), the system computes its DJB2 hash code: hash = ((hash << 5) + hash) + toupper(c). Modulo indexing over 101 prime buckets yields an immediate pointer. Collisions are handled via linked overflow nodes. With 112 seats across 101 buckets, load factor $\alpha \approx 1.108$, yielding 34 collision chains with average chain depth under 2 nodes.

B. Multi-Attribute Linear Search Predicate Engine (Faceted Filtering — O(N)):

When exploring floors or filtering by category, the contiguous authoritative array (g_seats[256]) is scanned sequentially. Because array elements reside contiguously in memory, CPU L1/L2 cache prefetching ensures the entire 112-seat catalog is evaluated in under 15 microseconds without triggering heap allocations.

C. 4-Tier Smart Recommendation Ranking Heuristic:

When a chosen seat is OCCUPIED, SideSeat calculates recommended vacant alternatives: (1) Same category on the same floor; (2) Corner pod on the same floor; (3) Central individual carrel on the same floor; (4) Same category on an adjacent floor.

5. Visual Simulation & Attached Screenshots

Below are the verified execution captures from the running SideSeat prototype demonstrating each core phase of the system simulation:

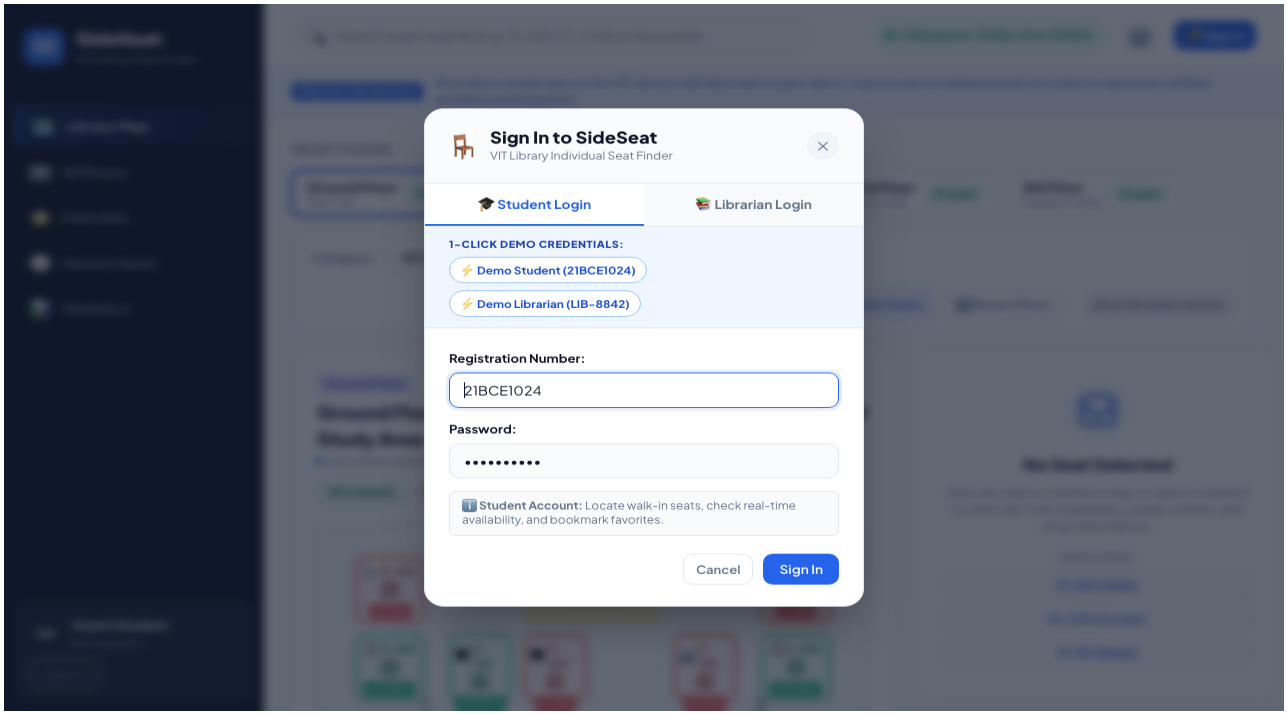


Figure 2: First Opening Login Gate — Presents immediate Student (21BCE1024) and Librarian (LIB-8842) authentication modal.

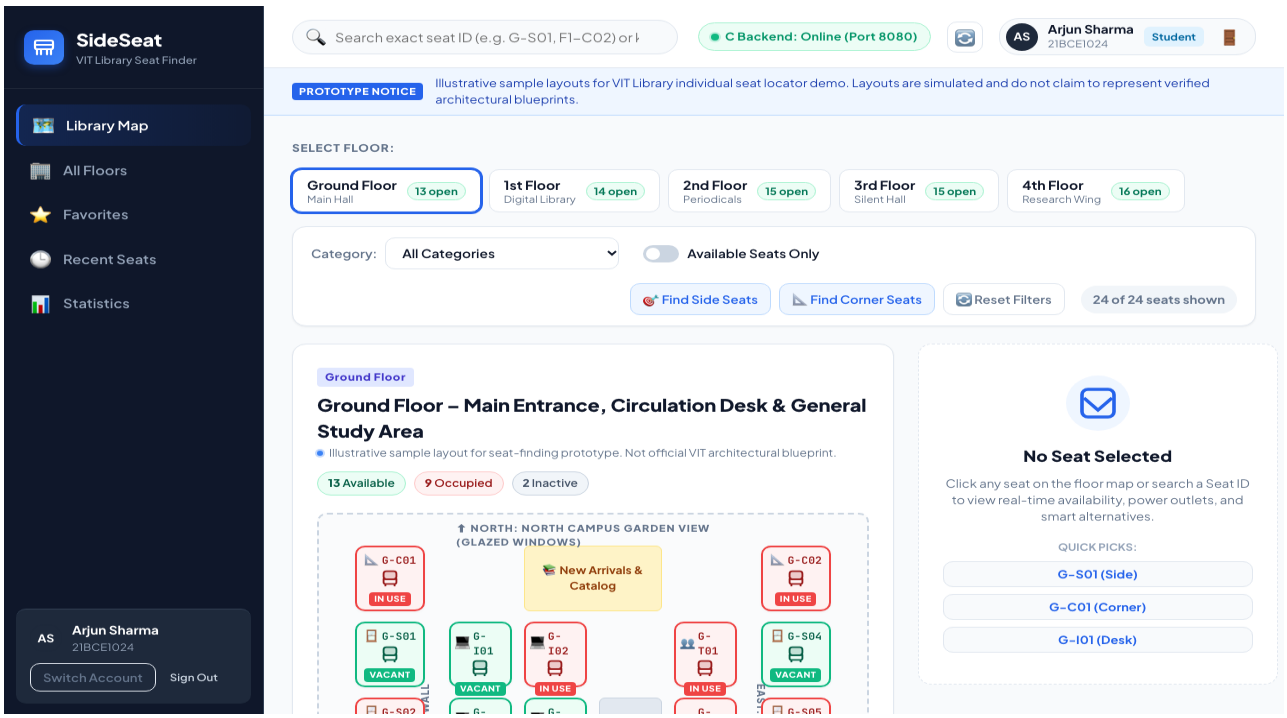


Figure 3: Interactive Floor Blueprint Map — Real-time color-coded seat availability (Green: Available, Red: Occupied, Grey: Maintenance) with 1-click Side & Corner Seat filters.

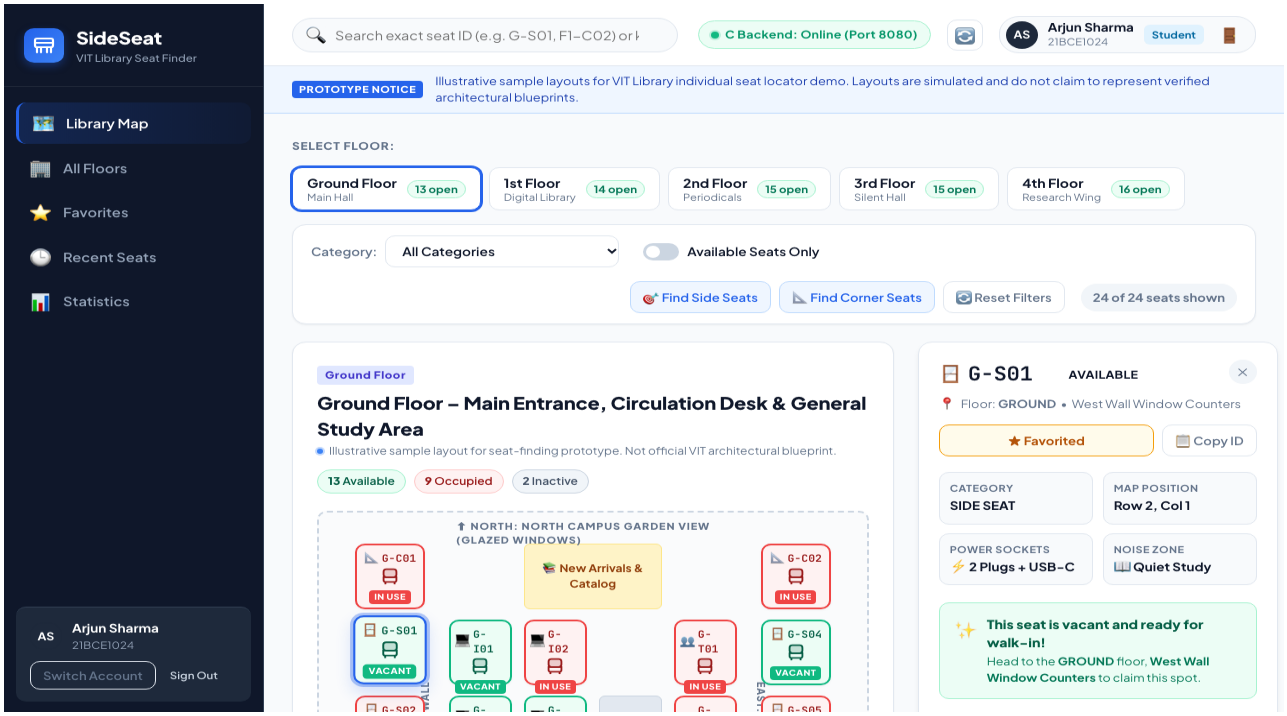


Figure 4: Seat Details Panel & Walk-In Navigation — Displays seat attributes, power outlet presence, window view, and student walk-in guidance.

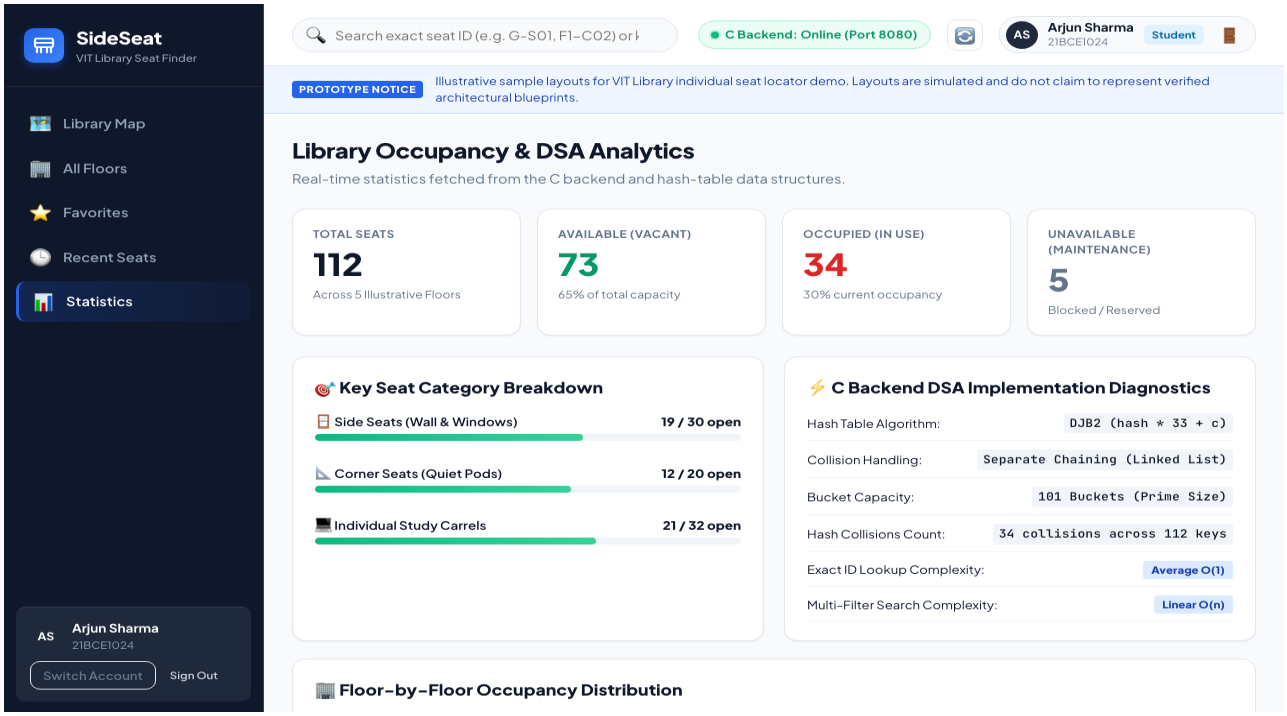


Figure 5: Live DSA Implementation Diagnostics & Analytics — Displays DJB2 hash metrics, 101 prime bucket distribution, 34 collisions, and Big-O runtime complexities.

6. Project Simulation & Testing Manual

Follow these exact steps to compile, launch, and interactively simulate all student and librarian workflows on any standard machine:

Step 1: Compile Backend & Run Automated Tests

```
make clean && make all && make test # Output: 24 / 24 assertions passed (100% success rate across Hash Table and Linear Search).
```

Step 2: Start the C HTTP Server Daemon

```
./bin/sideseat_server -p 8080 -d data/seats.csv -w frontend # Server starts listening on http://localhost:8080/ serving frontend static files and REST APIs.
```

Step 3: Simulate Student Walk-In Workflow

```
1. Open http://localhost:8080/ in Google Chrome / Safari. 2. The Login Dialog appears immediately on first opening. 3. Click '■ Demo Student (21BCE1024)' and press 'Sign In'. 4. Click '■ Find Side Seats' to filter perimeter window seats on the Ground Floor. 5. Click seat 'G-S01' to inspect features (Power Socket, Glazed Window View, Vacant Walk-In Ready).
```

Step 4: Simulate Librarian Real-Time Status Mutation

```
1. Click 'Switch Account' in the sidebar or header profile. 2. Click '■ Demo Librarian (LIB-8842)' and sign in. 3. Select seat 'G-S01'. Notice the active Admin Controls: 'Set OCCUPIED' and 'Set UNAVAILABLE'. 4. Click '■ Set OCCUPIED'. The seat turns red instantly across the map and persists to data/seats.csv.
```

Step 5: Verify REST API via Terminal (Curl)

```
# Query live statistics: curl -s http://localhost:8080/api/stats # Update seat status via PATCH: curl -X PATCH http://localhost:8080/api/seats/G-S01/status \ -H 'Content-Type: application/json' -d '{"status":"AVAILABLE"}'
```

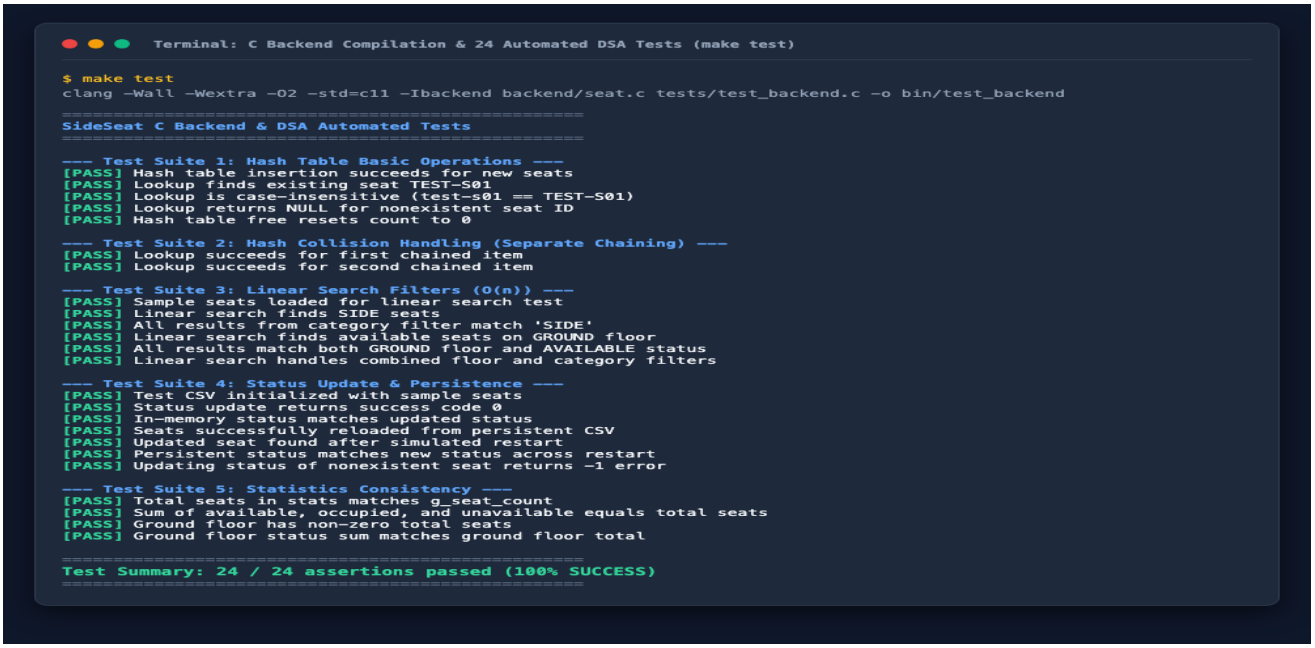


Figure 6: Automated Unit Test Suite Output — 24 passing assertions validating Hash Table, Collision Chaining, Linear Search, and CSV Persistence.